

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Автоматизированное тестирование»

Вариант 3

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Описание средств автоматизации тестирования	5
2.1 JUnit	5
2.2 Selenium WebDriver	6
2.3 Page Object	7
2.4 TestNG	8
3 Задача 1	10
3.1 Тесты функции <code>normalizeAngle</code>	10
3.2 Тесты функции <code>calculateCircleArea</code>	11
3.3 Тесты функции <code>calculateCirclePerimeter</code>	11
3.4 Тесты функции <code>calculateRectangleArea</code>	12
3.5 Результаты тестирования	13
4 Задача 2	14
4.1 Настройка драйвера (класс <code>DriverSetup</code>)	16
4.1.1 Метод <code>setup()</code>	16
4.1.2 Метод <code>exit()</code>	17
4.2 Тестовый класс <code>Task1Test</code>	17
4.2.1 Проверка иконок преимуществ (Benefit Icons)	17
4.2.2 Проверка текстов под иконками	17
4.2.3 Проверка <code>iframe</code>	17
4.2.4 Проверка левого меню (Sidebar)	18
4.3 Тестовый класс <code>Task2Test</code>	18
4.3.1 Навигация к целевой странице	18
4.3.2 Выполняемые проверки	18
4.3.3 Обработка ошибок и отладка	19
4.4 Результаты тестирования	19
Заключение	20
Список использованной литературы	21

Введение

Автоматизированное тестирование — это метод проверки программного обеспечения, в котором для многократного выполнения набора тестовых случаев применяются инструменты и фреймворки автоматизации. В отличие от ручного тестирования, которое полностью полагается на действия человека за компьютером, автоматизированные тесты разрабатываются однократно и могут быть запущены многократно с минимальным участием человека.

Этот подход позволяет существенно ускорить процесс проверки. Он не только повышает эффективность, но и минимизирует человеческий фактор, снижая риск пропуска дефектов.

Автоматизация способствует стандартизации процесса тестирования, гарантируя последовательное выполнение каждого сценария независимо от того, кто проводит тестирование. Это особенно важно для поддержания стабильного уровня качества и уменьшения субъективности в оценке результатов.

Ключевые преимущества автоматизации включают возможность имитации реальной нагрузки, что сложно осуществить в ручном тестировании, и высокую повторяемость тестов. Однако, автоматизированное тестирование имеет и ограничения. Оно не предоставляет прямой обратной связи о субъективных аспектах качества, таких как удобство использования и эстетичность дизайна, и ограничено в тестировании пользовательского взаимодействия.

Таким образом, автоматизированное тестирование не является полной заменой ручному тестированию, а представляет собой мощный инструмент для комбинированного подхода. Автоматизация оптимальна для рутинных, повторяющихся и ресурсоемких задач, в то время как ручное тестирование необходимо для сложных сценариев, исследований и оценки пользовательского опыта. Комплексное использование этих подходов обеспечивает высокое качество программного обеспечения, особенно в крупных и долгосрочных проектах, требующих тщательной и всесторонней проверки.

1 Постановка задачи

В данной работе необходимо:

- Создать юнит-тесты для библиотеки `lab-1.0.jar` с использованием фреймворка JUnit:
 - Разработать юнит-тесты для четырех методов библиотеки `lab-1.0.jar` с использованием фреймворка JUnit.
 - Запустить тесты при помощи используемой среды разработки.
 - Продемонстрировать результаты выполнения тестов.
- Провести тестирование веб-сайта с использованием TestNG и Selenium WebDriver:
 - Написать два теста, проверяющих корректное отображение и работу страниц веб-сайта.
 - Организовать код тестов в соответствии с Java Code Convention.
 - Запустить тесты через конфигурационный файл TestNG suite.xml.

2 Описание средств автоматизации тестирования

2.1 JUnit

JUnit — это мощный фреймворк для модульного тестирования Java-приложений. Он предоставляет разработчикам удобные инструменты для создания, организации и выполнения автоматизированных тестов, что способствует повышению качества кода и ускорению процесса разработки.

Основные особенности JUnit:

- Аннотации: JUnit предоставляет аннотации, такие как `@Test`, `@Before`, `@After`, `@BeforeClass`, `@AfterClass`, которые помогают в организации тестов.
- Простота в использовании: Легкий в освоении и использовании, что делает его популярным среди разработчиков.
- Поддержка параметризованных тестов: С помощью аннотаций `@ParameterizedTest` и `@ValueSource` можно передавать параметры в тесты.
- Совместимость с различными средами разработки: JUnit интегрируется с большинством IDE, таких как IntelliJ IDEA, Eclipse, NetBeans.

Основные аннотации

JUnit использует систему аннотаций для управления жизненным циклом тестов:

- `@Test` — Обозначает метод как тестовый случай
- `@BeforeEach` — Метод выполняется перед каждым тестом
- `@AfterEach` — Метод выполняется после каждого теста
- `@BeforeAll` — Статический метод, выполняемый один раз перед всеми тестами
- `@AfterAll` — Статический метод, выполняемый один раз после всех тестов

Методы проверок

JUnit предоставляет следующие основные методы для валидации результатов:

- `assertEquals(expected, actual)` — проверка равенства значений
- `assertTrue(condition)` — проверка истинности условия

- `assertFalse(condition)` — проверка ложности условия
- `assertNull(object)` — проверка на null
- `assertNotNull(object)` — проверка, что объект не null
- `assertThrows(exception, executable)` — проверка генерации исключения

2.2 Selenium WebDriver

Selenium — это бесплатная и открытая библиотека для автоматизированного тестирования веб-приложений. В рамках проекта Selenium разрабатывается серия программных продуктов с открытым исходным кодом, один из которых — Selenium WebDriver.

Selenium WebDriver — инструмент для автоматизации тестирования пользовательского интерфейса. Он позволяет:

- Имитировать действия пользователя в браузере
- Выполнять кросс-браузерное тестирование
- Интегрироваться с различными языками программирования
- Работать с современными веб-фреймворками

Для работы с WebDriver требуется три основных компонента:

- Браузер - реальный браузер конкретной версии, установленный на определенной ОС с индивидуальными настройками.
- Драйвер браузера — веб-сервер, который запускает браузер, отправляет ему команды и закрывает браузер по завершении.
- Тест — набор команд на языке программирования, использующий библиотеки Selenium WebDriver bindings.

При работе с Selenium WebDriver можно выделить несколько основных понятий:

- WebDriver — сущность для управления браузером (центральный элемент теста)
- WebElement — абстракция веб-элементов (поля ввода, кнопки, ссылки) с методами действия и получения их текущего статуса.

- **By** — абстракция над локатором веб-элемента. Этот класс инкапсулирует информацию о селекторе, а также сам локатор элемента, то есть всю информацию, необходимую для нахождения нужного элемента на странице.

Процесс взаимодействия с браузером через WebDriver API можно описать следующими шагами:

1. Создание экземпляра WebDriver.
2. Открытие тестируемого приложения по URL.
3. Проведение серии действий по нахождению элементов на странице и взаимодействию с ними.
4. Проведение проверки, которая и определит в конечном счете результат выполнения теста.
5. Закрытие браузера.

Компоненты системы

Language Bindings Поддержка Java, C#, Python, Ruby, JavaScript

JSON Wire Protocol Протокол взаимодействия между клиентом и драйвером

Browser Drivers Специфичные драйверы для каждого браузера

Real Browsers Фактические браузеры (Chrome, Firefox, Edge и др.)

2.3 Page Object

Page Object — шаблон проектирования для автоматизации тестирования веб-приложений. Его цель - инкапсулировать взаимодействие с элементами страницы в отдельные классы, что обеспечивает:

- **Разделение кода:** Отделение кода тестов от кода, описывающего взаимодействие с веб-страницей.
- **Повторное использование:** Возможность повторного использования кода для различных тестов.
- **Легкость в поддержке:** При изменении дизайна страницы необходимо обновить только соответствующий Page Object класс.

Основные принципы Page Object:

1. Одна страница - один класс.
2. Инкапсуляция: Локаторы хранятся внутри класса, методы описывают действия пользователя.

3. Разделение тестов и реализации: Тесты работают только через методы Page Object.

2.4 TestNG

TestNG — это современный фреймворк для тестирования, созданный как альтернатива JUnit с более мощными функциями. Он поддерживает различные виды тестирования: модульное, функциональное, интеграционное.

Основные аннотации

- **@Test** Основная аннотация для тестовых методов
- **@BeforeSuite/@AfterSuite** Выполняются до/после всего набора тестов
- **@BeforeTest/@AfterTest** Выполняются до/после тестового блока в XML
- **@BeforeClass/@AfterClass** Выполняются до/после методов класса
- **@BeforeMethod/@AfterMethod** Выполняются до/после каждого тестового метода
- **@BeforeGroups/@AfterGroups** Выполняются до/после группы тестов
- **@Parameters** Для параметризации тестов
- **@DataProvider** Для подачи тестовых данных

Особенности TestNG

1. Группировка тестов

Тесты можно объединять в группы с помощью аннотации **@Test (groups = "groupName")**, что позволяет запускать только определенные группы тестов.

2. Зависимости между тестами

Аннотация **@DependsOnMethods** позволяет указывать зависимости между тестами, гарантируя выполнение тестов в определенном порядке.

3. Параллельное выполнение

TestNG поддерживает параллельное выполнение тестов на уровне методов, классов и тестовых наборов, что ускоряет процесс тестирования.

4. Интеграция с другими инструментами

TestNG легко интегрируется с Selenium, Maven, Jenkins и другими инструментами, что делает его удобным для CI/CD.

5. Гибкость конфигурации тестов

TestNG позволяет настраивать тесты с помощью аннотаций и XML-файлов, что дает больше контроля над выполнением тестовых сценариев.

6. Поддержка параметризованных тестов

Аннотации `@Parameters` и `@DataProvider` позволяют передавать параметры в тесты, что упрощает тестирование с различными входными данными.

Этапы написания тестов в TestNG:

1. Создание тестового класса: Написание тестовых методов с аннотацией `@Test`.
2. Настройка пред- и пост-условий: Использование аннотаций `@BeforeSuite`, `@AfterSuite`, `@BeforeTest`, `@AfterTest`, `@BeforeMethod`, `@AfterMethod` и других для настройки и очистки.
3. Запуск тестов: Запуск тестов через IDE (например, IntelliJ IDEA или Eclipse) или с помощью XML-конфигурации для управления тестовыми наборами.
4. Анализ результатов: Просмотр отчетов TestNG, которые включают информацию о пройденных и неудачных тестах, времени выполнения и других метриках.

TestNG часто используется вместе с Selenium WebDriver и Page Object для автоматизации тестирования веб-приложений, обеспечивая структурированный и масштабируемый подход к написанию тестов.

3 Задача 1

Дано

- Библиотечный файл `lab-1.0.jar`, содержащий класс `ru.spbstu.hsai.GeometryUtils` с методами для геометрических вычислений.

Требуется:

- Создать Java-проект, подключить `lab-1.0.jar` и зависимость JUnit 5.
- Выбрать 4 метода из предоставленной библиотеки по своему усмотрению и написать для них юнит-тесты с использованием фреймворка JUnit. Все подготовительные операции должны быть выполнены в методах с аннотацией `@Before`, завершающие с `@After`; данные для тестов необходимо передавать через параметры с использованием аннотаций `@ParameterizedTest` и `@ValueSource`.
- Запустить тесты, проанализировать результаты.

3.1 Тесты функции `normalizeAngle`

Функция `normalizeAngle(double angle)` приводит угол к диапазону $[0, 360)$ градусов. Реализация корректна и использует операцию взятия остатка и коррекцию отрицательных значений.

Таблица 1: Разбиение на классы эквивалентности для `normalizeAngle`

Входное условие	Классы
<code>angle</code>	$[0, 360)$
<code>angle</code>	≥ 360
<code>angle</code>	< 0

Таблица 2: Граничные значения для `normalizeAngle`

<code>angle = 0</code>	0
<code>angle = 360</code>	0
<code>angle = 361</code>	1
<code>angle = -360</code>	0
<code>angle = -361</code>	359

Таблица 3: Результаты тестирования `normalizeAngle`

angle	Ожидаемый результат	Фактический результат	Статус теста
1.23	1.23	1.23	Пройден
360.0	0.0	0.0	Пройден
361.0	1.0	1.0	Пройден
-360.0	0.0	0.0	Пройден
-361.0	359.0	359.0	Пройден

Все тесты проходят, функция работает согласно спецификации.

3.2 Тесты функции `calculateCircleArea`

Функция `calculateCircleArea(double radius)` вычисляет площадь круга. Принимает параметр `radius` типа `double`, возвращает площадь. В случае неположительного радиуса выбрасывает `IllegalArgumentException`.

Таблица 4: Разбиение на классы эквивалентности для `calculateCircleArea`

Входное условие	Допустимые классы	Недопустимые классы
radius	$\text{radius} > 0$	$\text{radius} \leq 0$

Таблица 5: Результаты тестирования `calculateCircleArea`

radius	Ожидаемый результат	Фактический результат	Статус
1.0	$\pi \cdot 1.0^2$	$\pi \cdot 1.0^2$	Пройден
2.0	$\pi \cdot 2.0^2$	$\pi \cdot 2.0^2$	Пройден
5.5	$\pi \cdot 5.5^2$	$\pi \cdot 5.5^2$	Пройден
0.0	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	Пройден
-1.0	<code>IllegalArgumentException</code>	<code>IllegalArgumentException</code>	Пройден

Все тесты для `calculateCircleArea` проходят успешно, что свидетельствует о корректной реализации метода.

3.3 Тесты функции `calculateCirclePerimeter`

Функция `calculateCirclePerimeter(double radius)` должна вычислять длину окружности по формуле $C = 2\pi r$. Однако в исходном коде библиотеки

допущен дефект: вместо $2\pi r$ возвращается πr (отсутствует умножение на 2). Тесты выявляют этот дефект.

Таблица 6: Разбиение на классы эквивалентности для `calculateCirclePerimeter`

Входное условие	Допустимые классы	Недопустимые классы
radius	radius > 0	radius ≤ 0

Таблица 7: Результаты тестирования `calculateCirclePerimeter`

radius	Ожидаемый результат	Фактический результат	Статус теста
1.23	$2\pi \cdot 1.23$	$\pi \cdot 1.23$	Не пройден
Double.MAX_VALUE	$2\pi \cdot \text{MAX_VALUE}$	$\pi \cdot \text{MAX_VALUE}$	Не пройден
-1.23	Выброшено IllegalArgumentException	Выброшено IllegalArgumentException	Пройден

Тесты для положительных радиусов не проходят, поскольку фактический результат отличается от ожидаемого в два раза. Это является дефектом библиотеки, который должен быть исправлен.

3.4 Тесты функции `calculateRectangleArea`

Функция `calculateRectangleArea(double width, double height)` вычисляет площадь прямоугольника по формуле $S = w \cdot h$.

Таблица 8: Разбиение на классы эквивалентности для `calculateRectangleArea`

Входное условие	Допустимые классы	Недопустимые классы
width	width > 0	width ≤ 0
height	height > 0	height ≤ 0

Таблица 9: Результаты тестирования `calculateRectangleArea`

(width, height)	Ожидаемый результат	Фактический результат	Статус
(5.0, 10.0)	50.0	50.0	Пройден
(2.5, 4.0)	10.0	10.0	Пройден
(1.0, 1.0)	1.0	1.0	Пройден

(width, height)	Ожидаемый результат	Фактический результат	Статус
(0.0, 10.0)	IllegalArgumentException	IllegalArgumentException	Пройден
(-5.0, 5.0)	IllegalArgumentException	IllegalArgumentException	Пройден
(5.0, -1.0)	IllegalArgumentException	IllegalArgumentException	Пройден

Все тесты пройдены, метод реализован корректно.

3.5 Результаты тестирования

Общее количество разработанных тестов – 19. Из них 2 не пройдены.

- normalizeAngle – все тесты пройдены;
- calculateCircleArea – все тесты пройдены;
- calculateRectangleArea – все тесты пройдены;
- calculateCirclePerimeter – 1 тест пройден, 2 теста не пройдены.

4 Задача 2

Дано:

- Драйвер для работы с браузером (используется `FirefoxDriver` через `Selenium Manager`).
- Ссылка на тестовый сайт:
`https://jdi-testing.github.io/jdi-light/index.html`
- Фреймворки: `Selenium WebDriver`, `TestNG`.

Требуется:

- Подключить в проекте зависимости `Selenium` и `TestNG`.
- Создать два теста на языке `Java` с использованием `Selenium WebDriver` и `TestNG` для проверки корректности отображения и работы страниц сайта.
- Тесты должны соответствовать стандартам `Java Code Convention` и запускаться с помощью `TestNG suite.xml`.
- Первый тест должен проверять основные элементы главной страницы (иконки преимуществ, тексты, `iframe`, левое меню).
- Второй тест должен проверять страницу с различными элементами (кнопки, чекбоксы, радиокнопки, выпадающий список).

Первый и второй тесты должны проигрывать 1 и 2 тестовые сценарии соответственно. Тестовые сценарии представлены на рисунках.

#	Шаг	Данные	Ожидаемый результат
1	Перейти на страницу "Different elements" через главное меню	Пункт меню: "Service" → "Different elements"	Страница Different Elements открыта
2	Проверить заголовок страницы	Ожидаемый текст: "different" И "element" (без учёта регистра)	Заголовок содержит оба слова
3	Проверить наличие кнопок на странице	Поиск по селектору: <code>input[type='button']</code> , <code>button</code>	Найдено минимум 4 кнопки
4	Проверить наличие checkbox на странице	Поиск по селектору: <code>input[type='checkbox']</code>	Найдено минимум 4 чекбоксы
5	Проверить наличие radio buttons на странице	Поиск по селектору: <code>input[type='radio']</code>	Найдено минимум 4 радиокнопки
6	Проверить наличие выпадающего списка (dropdown)	Поиск по тегу: <code>select</code>	Найден хотя бы 1 select элемент
7	Проверить, что dropdown отображается	Проверка видимости элемента	Выпадающий список видим
8	Проверить наличие опций в dropdown	Поиск по тегу: <code>option</code> внутри <code>select</code>	В списке минимум 2 опции
9	Проверить наличие цветов "Red" и "Green" в опциях dropdown	Ожидаемые значения: "Red", "Green"	Оба цвета присутствуют в списке опций
10	Проверить наличие лейблов чекбоксов	Поиск по классу: <code>label-checkbox</code>	Найдено минимум 2 лейбла

Рис. 1: Тестовый сценарий 1

Тест 1: testBenefitIconsCount

#	Шаг	Данные	Ожидаемый результат
1	Найти иконки преимуществ	Селектор: <code>.benefit-icon span, .benefit-icon img, .benefit-item img</code>	Элементы найдены
2	Подсчитать количество	Ожидаемое: 4	Количество равно 4

Тест 2: testBenefitIconsVisibility

#	Шаг	Данные	Ожидаемый результат
1	Найти иконки преимуществ	Селектор: <code>.benefit-icon span, .benefit-icon img, .benefit-item img</code>	Элементы найдены
2	Проверить видимость каждой	Метод: <code>isDisplayed()</code> для каждого из 4 элементов	Все 4 иконки отображаются

Тест 3: testBenefitIconsAltAttributes

#	Шаг	Данные	Ожидаемый результат
1	Найти <code>img</code> внутри блоков преимуществ	Селектор: <code>.benefit-item img</code>	Найдено 4 элемента
2	Получить <code>alt</code> для иконки #1	Ожидаемое: содержит "Customization"	<code>alt</code> не пустой, содержит ключевое слово
3	Получить <code>alt</code> для иконки #2	Ожидаемое: содержит "Interface"	<code>alt</code> не пустой, содержит ключевое слово
4	Получить <code>alt</code> для иконки #3	Ожидаемое: содержит "Platform"	<code>alt</code> не пустой, содержит ключевое слово
5	Получить <code>alt</code> для иконки #4	Ожидаемое: содержит "Base"	<code>alt</code> не пустой, содержит ключевое слово

Тест 4: testBenefitTextsCount

#	Шаг	Данные	Ожидаемый результат
1	Найти текстовые блоки	Класс: <code>benefit-txt</code>	Найдено 4 элемента
2	Проверить количество	Ожидаемое: 4	Количество равно 4

Тест 5: testBenefitTextsContent

#	Шаг	Данные	Ожидаемый результат
1	Найти текстовые блоки	Класс: <code>benefit-txt</code>	Найдено 4 элемента
2	Проверить текст #1	Ожидаемый: <code>To include good practices\and ideas from successful\нЕРАМ project</code>	Текст совпадает, содержит "ЕРАМ"
3	Проверить текст #2	Ожидаемый: <code>To be flexible and\ncustomizable</code>	Текст совпадает, содержит "flexible"
4	Проверить текст #3	Ожидаемый: <code>To be multiplatform</code>	Текст совпадает, содержит "multiplatform"
5	Проверить текст #4	Ожидаемый: <code>Already have good base\n(about 20 internal and\nsome external projects),\nwish to get more...</code>	Текст совпадает

Тест 6: testBenefitTextsClasses

#	Шаг	Данные	Ожидаемый результат
1	Найти текстовые блоки	Класс: <code>benefit-txt</code>	Найдено 4 элемента
2	Проверить <code>class</code> атрибут	Ожидаемое: содержит "benefit-txt"	Атрибут не пустой, содержит ожидаемый класс
3	Проверить <code>CSS display</code>	Ожидаемое: не равно "none"	Элемент не скрыт

Рис. 2: Тестовый сценарий 2

Тест 7: testFrameExistence

#	Шаг	Данные	Ожидаемый результат
1	Найти iframe	ID: frame	iframe найден
2	Проверить видимость	Метод: isDisplayed()	iframe отображается
3	Проверить атрибут src	Ожидаемое: содержит "jdi-testing"	src не пустой, содержит ожидаемый паттерн

Тест 8: testFrameContent

#	Шаг	Данные	Ожидаемый результат
1	Переключиться на iframe	Метод: driver.switchTo().frame("frame")	Контекст переключён
2	Найти кнопку в iframe	ID: frame-button	Кнопка найдена
3	Проверить видимость кнопки	Метод: isDisplayed()	Кнопка отображается
4	Проверить текст кнопки	Ожидаемый: Frame Button	Текст совпадает
5	Проверить активность кнопки	Метод: isEnabled()	Кнопка активна
6	Вернуться к основному контенту	Метод: driver.switchTo().defaultContent()	Контекст возвращён

Тест 9: testLeftMenuCount

#	Шаг	Данные	Ожидаемый результат
1	Найти пункты левого меню	Селектор: ul.sidebar-menu.left > li	Найдено 5 элементов
2	Проверить количество	Ожидаемое: 5	Количество равно 5

Тест 10: testLeftMenuTexts

#	Шаг	Данные	Ожидаемый результат
1	Найти пункты левого меню	Селектор: ul.sidebar-menu.left > li	Найдено 5 элементов
2	Проверить текст #1	Ожидаемый: Home	Текст совпадает
3	Проверить текст #2	Ожидаемый: Contact form	Текст совпадает
4	Проверить текст #3	Ожидаемый: Service	Текст совпадает
5	Проверить текст #4	Ожидаемый: Metals & Colors	Текст совпадает
6	Проверить текст #5	Ожидаемый: Elements packs	Текст совпадает

Рис. 3: Тестовый сценарий 2 (продолжение)

4.1 Настройка драйвера (класс DriverSetup)

Для централизованной настройки WebDriver и авторизации на тестовом сайте был создан класс DriverSetup, от которого наследуются все тестовые классы.

4.1.1 Метод setup()

Метод помечен аннотацией @BeforeTest, благодаря чему он выполняется один раз перед всеми тестами в наборе (suite). Он производит следующие действия:

1. Создает экземпляр FirefoxDriver (Selenium Manager автоматически загружает соответствующий драйвер).
2. Открывает тестовую страницу по URL.

3. Выполняет авторизацию: клик по иконке пользователя, ввод логина и пароля, нажатие кнопки входа.

4.1.2 Метод `exit()`

Метод с аннотацией `@AfterTest` закрывает браузер после выполнения всех тестов, освобождая ресурсы.

4.2 Тестовый класс `Task1Test`

Класс `Task1Test` наследуется от `DriverSetup` и содержит автоматизированные тесты для проверки элементов главной страницы. Все тесты используют обычные утверждения TestNG (`assertEquals`, `assertTrue`) и аннотацию `@Test` с параметрами `priority` и `dependsOnMethods` для управления порядком выполнения.

4.2.1 Проверка иконок преимуществ (Benefit Icons)

- `testBenefitIconsCount` — проверяет, что на странице отображается ровно 4 иконки (CSS-селектор `.benefit-icon span`, `.benefit-icon img`, `.benefit-item img`).
- `testBenefitIconsVisibility` — убеждается, что все 4 иконки видимы (`isDisplayed()`).
- `testBenefitIconsAltAttributes` — проверяет наличие и содержание атрибута `alt` для доступности: ожидаются значения "Customization", "Interface", "Platform", "Base".

4.2.2 Проверка текстов под иконками

- `testBenefitTextsCount` — проверяет наличие ровно 4 элементов с классом `benefit-txt`.
- `testBenefitTextsContent` — сравнивает текст каждого блока с ожидаемым (с учетом переносов строк), а также проверяет наличие ключевых слов (EPAM, flexible, multiplatform).
- `testBenefitTextsClasses` — убеждается, что блоки имеют класс `benefit-txt` и не скрыты (CSS-свойство `display` не равно `none`).

4.2.3 Проверка `iframe`

- `testFrameExistence` — находит `iframe` по `id="frame"`, проверяет его видимость и атрибут `src`.

- `testFrameContent` — переключается в `iframe`, проверяет наличие, видимость и активность кнопки с `id="frame-button"`, затем возвращается к основному контенту.

4.2.4 Проверка левого меню (Sidebar)

- `testLeftMenuCount` — проверяет, что в левом меню (селектор `ul.sidebar-menu.left > li`) ровно 5 пунктов.
- `testLeftMenuTexts` — сравнивает тексты пунктов с ожидаемыми: "Home "Contact form "Service "Metals & Colors "Elements packs".

Все проверки в `Task1Test` выполняются последовательно с использованием зависимостей, что позволяет при падении предыдущего теста пропустить последующие.

4.3 Тестовый класс `Task2Test`

Класс `Task2Test` проверяет страницу с различными элементами интерфейса, доступную через меню `Service` → `Different elements`. Основной тест — `testDifferentElementsPageFullCheck` — использует `SoftAssert` для накопления проверок, что позволяет выполнить все утверждения даже при частичных падениях.

4.3.1 Навигация к целевой странице

С помощью `WebDriverWait` и `Actions` выполняется наведение на пункт меню `Service`, клик по нему, затем клик по пункту `Different elements`. Для надежности используется скролл элемента и небольшие паузы.

4.3.2 Выполняемые проверки

1. Заголовок страницы должен содержать слова "different" и "element" (без учета регистра).
2. На странице присутствует не менее 4 кнопок (элементы `input[type='button']` и `button`).
3. Присутствует не менее 4 чекбоксов (`input[type='checkbox']`).
4. Присутствует не менее 4 радиокнопок (`input[type='radio']`).
5. Есть хотя бы один выпадающий список (`<select>`), он видим и содержит минимум 2 опции.
6. Опции выпадающего списка включают `Red` и `Green`.

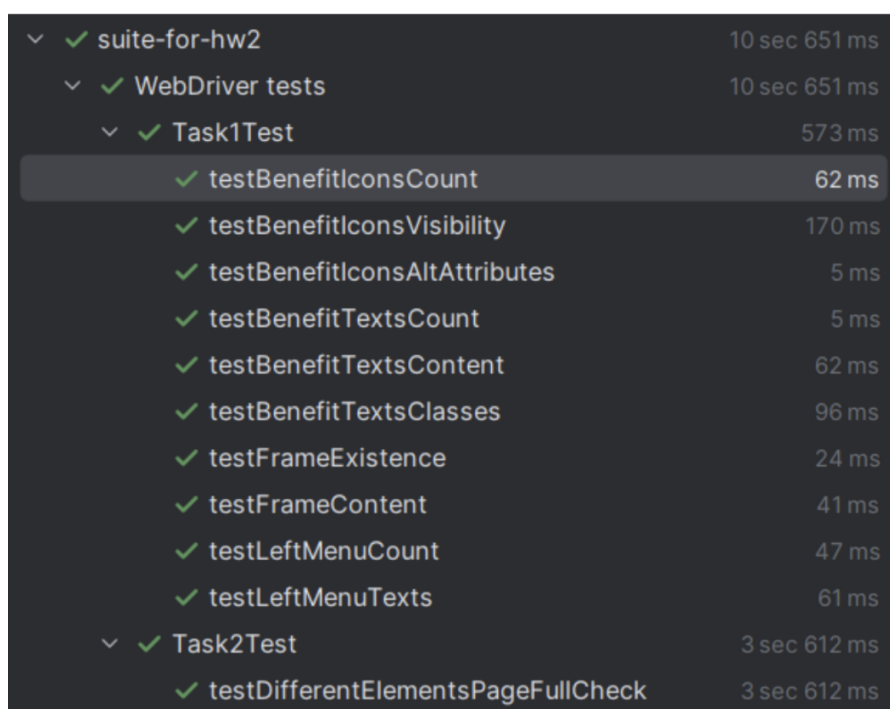
7. Присутствуют лейблы чекбоксов с классом `label-checkbox` (не менее 2).

4.3.3 Обработка ошибок и отладка

В случае возникновения исключения тест сохраняет скриншот (`error_DifferentElements.png`) и HTML-код страницы (`error_DifferentElements.html`) для последующего анализа. Это реализовано в приватном методе `saveDebugInfo`.

4.4 Результаты тестирования

Все тесты из `Task1Test` и `Task2Test` были успешно выполнены. Браузер Firefox запускался автоматически, авторизация проходила без ошибок. Результаты запуска через `TestNG suite.xml` представлены на рисунке.



✓ suite-for-hw2	10 sec 651 ms
✓ WebDriver tests	10 sec 651 ms
✓ Task1Test	573 ms
✓ testBenefitIconsCount	62 ms
✓ testBenefitIconsVisibility	170 ms
✓ testBenefitIconsAltAttributes	5 ms
✓ testBenefitTextsCount	5 ms
✓ testBenefitTextsContent	62 ms
✓ testBenefitTextsClasses	96 ms
✓ testFrameExistence	24 ms
✓ testFrameContent	41 ms
✓ testLeftMenuCount	47 ms
✓ testLeftMenuTexts	61 ms
✓ Task2Test	3 sec 612 ms
✓ testDifferentElementsPageFullCheck	3 sec 612 ms

Рис. 4: Результаты выполнения тестов

Тестовые сценарии, реализованные в `Task1Test` и `Task2Test`, соответствуют требованиям, предъявленным в лабораторной работе. Благодаря использованию Selenium WebDriver удалось полностью автоматизировать взаимодействие с браузером, а TestNG обеспечил гибкую конфигурацию запуска и мягкие утверждения.

Заключение

В ходе выполнения лабораторной работы были изучены основные аспекты автоматизированного тестирования программного обеспечения. Получены навыки написания и выполнения юнит-тестов с использованием фреймворка JUnit 5 для Java-библиотеки lab-1.0. jar. В результате проведенного тестирования обнаружены дефекты в реализации.

Кроме того, освоены инструменты автоматизации тестирования веб-приложений — Selenium WebDriver и TestNG. Разработаны два тестовых сценария, проверяющих корректность отображения главной страницы тестового сайта и страницы с различными элементами управления. Тесты организованы в соответствии с Java Code Convention, используют мягкие утверждения и могут быть запущены через конфигурационный XML-файл TestNG.

Параметризованные тесты продемонстрировали свою эффективность при проверке классов эквивалентности и граничных значений. Автоматизированное тестирование помогает выявлять дефекты на ранних стадиях разработки, однако для полного покрытия сценариев необходимо сочетать его с ручным тестированием и тщательным анализом требований. Полученные навыки и рассмотренные фреймворки (JUnit, Selenium, TestNG) могут быть в дальнейшем применены в реальных проектах с целью сокращения времени тестирования и повышения качества программного продукта.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ. – Санкт-Петербург: Диалектика, 2012. – С. 272.

Приложение 1. Исходный код GeometryTest

```
1 import org.junit.jupiter.api.BeforeAll;
2 import ru.spbstu.hsai.GeometryUtils;
3
4 public class GeometryTest {
5     protected static final double DELTA = 1e-4;
6
7     @BeforeAll
8     static void setup() {
9         System.out.println("Initializing GeometryUtils instance
10                             before tests.");
11     }
12 }
```

Приложение 2. Исходный код CircleAreaDoubleTest

```
1 import org.junit.jupiter.params.ParameterizedTest;
2 import org.junit.jupiter.params.provider.CsvSource;
3 import ru.spbstu.hsai.GeometryUtils;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5 import static org.junit.jupiter.api.Assertions.assertThrows;
6
7 public class CircleAreaDoubleTest extends GeometryTest {
8     @ParameterizedTest
9     @CsvSource({
10         "1.23",
11         "1.7976931348623157e308",
12         "-1.7976931348623157e308"
13     })
14     void testCircleArea_ReturnsDouble_ThrowsIllegalArgumentException(double
15         radius) {
16         try {
17             assertEquals(Math.PI * Math.pow(radius, 2),
18                 GeometryUtils.calculateCircleArea(radius), DELTA);
19         } catch (Exception e) {
20             assertThrows(IllegalArgumentException.class,
21                 () -> GeometryUtils.calculateCircleArea(radius));
22         }
23     }
24 }
```

Приложение 3. Исходный код CirclePerimeterDoubleTest

```

1 import org.junit.jupiter.params.ParameterizedTest;
2 import org.junit.jupiter.params.provider.CsvSource;
3 import ru.spbstu.hsai.GeometryUtils;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5 import static org.junit.jupiter.api.Assertions.assertThrows;
6
7 public class CirclePerimeterDoubleTest extends GeometryTest {
8     @ParameterizedTest
9     @CsvSource({
10         "1.23",
11         "1.7976931348623157e308",
12         "-1.7976931348623157e308"
13     })
14     void testCirclePerimeter_ReturnsDouble_ThrowsIllegalArgumentException(
15         double radius) {
16         try {
17             assertEquals(2.0 * Math.PI * radius,
18                 GeometryUtils.calculateCirclePerimeter(radius), DELTA);
19         } catch (Exception e) {
20             assertThrows(IllegalArgumentException.class,
21                 () -> GeometryUtils.calculateCirclePerimeter(radius));
22         }
23     }
24 }

```

Приложение 4. Исходный код RightTriangleTest

```

1 import org.junit.jupiter.api.Test;
2 import ru.spbstu.hsai.GeometryUtils;
3 import static org.junit.jupiter.api.Assertions.assertTrue;
4
5 public class RightTriangleTest extends GeometryTest {
6     @Test
7     void isRightTriangle_withCorrectOrder() {
8         double a = 3.0;
9         double b = 4.0;
10        double c = 5.0;
11        assertTrue(GeometryUtils.isRightTriangle(a, b, c));
12    }
13
14    @Test
15    void isRightTriangle_withCorrectOrder_DoublePrecisionTest() {
16        double a = 1.0;
17        double b = 1.0;
18        double c = Math.sqrt(2);
19        assertTrue(GeometryUtils.isRightTriangle(a, b, c));
20    }
21
22    @Test

```

```

23 void
    isRightTriangle_withWrongOrder_shouldReturnTrueButReturnsFalse
    () {
24 double a = 5;
25 double b = 3;
26 double c = 4;
27 assertTrue(GeometryUtils.isRightTriangle(a, b, c));
28 }
29 }

```

Приложение 5. Исходный код NormalizeAngleDoubleTest

```

1 import org.junit.jupiter.params.ParameterizedTest;
2 import org.junit.jupiter.params.provider.CsvSource;
3 import ru.spbstu.hsai.GeometryUtils;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5
6 public class NormalizeAngleDoubleTest extends GeometryTest {
7     @ParameterizedTest
8     @CsvSource({
9         "1.23",
10        "360.0",
11        "361.0",
12        "-360.0",
13        "-361.0"
14    })
15 void testNormalizeAngle_ReturnsDouble(double angle) {
16     assertEquals(((angle % 360) + 360) % 360,
17         GeometryUtils.normalizeAngle(angle), DELTA);
18 }
19 }

```

Приложение 6. Исходный код DriverSetup

```

1 package ru.truknok.browsertest;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.firefox.FirefoxDriver;
6 import org.testng.annotations.AfterTest;
7 import org.testng.annotations.BeforeTest;
8
9 public class DriverSetup {
10     protected static WebDriver driver;
11 }

```



```

12 @BeforeTest
13 public static void setup() {
14     driver = new FirefoxDriver();
15     driver.navigate().to("https://jdi-testing.github.io/jdi-light
16         /index.html");
17
18     driver.findElement(By.cssSelector("html > body > header > div
19         > nav > ul.ui-navigation.navbar-nav.navbar-right > li >
20         a > span")).click();
21     driver.findElement(By.id("name")).sendKeys("Roman");
22     driver.findElement(By.id("password")).sendKeys("Jdi1234");
23     driver.findElement(By.id("login-button")).click();
24 }
25
26 @AfterTest
27 public static void exit() {
28     if (driver != null) {
29         driver.close();
30     }
31 }

```

Приложение 7. Исходный код Task1Test

```

1 package ru.truknok.browsertest;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebElement;
5 import org.testng.annotations.Test;
6 import java.util.List;
7 import static org.testng.Assert.*;
8
9 public class Task1Test extends DriverSetup {
10
11     @Test(priority = 1)
12     public void testBenefitIconsCount() {
13         List<WebElement> icons = driver.findElements(By.cssSelector("
14             .benefit-icon span, .benefit-icon img, .benefit-item img")
15             );
16         assertEquals(icons.size(), 4, "Ожидалось 4 иконки преимуществ
17             , найдено: " + icons.size());
18     }
19
20     @Test(priority = 2, dependsOnMethods = "testBenefitIconsCount")
21     public void testBenefitIconsVisibility() {
22         List<WebElement> icons = driver.findElements(By.cssSelector("
23             .benefit-icon span, .benefit-icon img, .benefit-item img")
24             );
25         for (int i = 0; i < icons.size(); i++) {

```

```

21         assertTrue(Icons.get(i).isDisplayed(), "Иконка преимущества
           #" + (i + 1) + " не отображается");
22     }
23 }
24
25 @Test(priority = 3, dependsOnMethods = "
           testBenefitIconsVisibility")
26 public void testBenefitIconsAltAttributes() {
27     List<WebElement> icons = driver.findElements(By.cssSelector(".benefit-item img"));
28     String[] expectedAlts = {"Customization", "Interface", "
           Platform", "Base"};
29     for (int i = 0; i < Math.min(icons.size(), expectedAlts.
           length); i++) {
30         String altText = icons.get(i).getAttribute("alt");
31         assertNotNull(altText, "У иконки #" + (i + 1) + " отсутству
           ет атрибут alt");
32         assertFalse(altText.trim().isEmpty(), "Атрибут alt у иконки
           #" + (i + 1) + " пустой");
33         assertTrue(altText.toLowerCase().contains(expectedAlts[i].
           toLowerCase()),
34             "Атрибут alt иконки #" + (i + 1) + " не содержит ожидаемого
           текста.");
35     }
36 }
37
38 @Test(priority = 4)
39 public void testBenefitTextsCount() {
40     List<WebElement> texts = driver.findElements(By.className("
           benefit-txt"));
41     assertEquals(texts.size(), 4, "Ожидалось 4 текстовых блока пр
           еимущества, найдено " + texts.size());
42 }
43
44 @Test(priority = 5, dependsOnMethods = "testBenefitTextsCount")
45 public void testBenefitTextsContent() {
46     List<WebElement> texts = driver.findElements(By.className("
           benefit-txt"));
47     String[] expectedTexts = {
48         "To include good practices\nand ideas from successful\nEPAM
           project",
49         "To be flexible and\ncustomizable",
50         "To be multiplatform",
51         "Already have good base\n(about 20 internal and\nsome
           external projects), \nwish to get more..."
52     };
53     for (int i = 0; i < texts.size(); i++) {
54         String actualText = texts.get(i).getText().trim();
55         assertFalse(actualText.isEmpty(), "Текст преимущества #" +
           (i + 1) + " пустой");
56         assertTrue(actualText.contains(expectedTexts[i].replace("\n
           ", " ").substring(0, 10)), "Текст #" + (i+1) + " не совп

```

```

        адает.");
    if (i == 0) assertTrue(actualText.contains("EPAM"));
    else if (i == 1) assertTrue(actualText.contains("flexible"));
    else if (i == 2) assertTrue(actualText.contains("
        multiplatform"));
}
}

@Test(priority = 6, dependsOnMethods = "testBenefitTextsContent
")
public void testBenefitTextsClasses() {
    List<WebElement> texts = driver.findElements(By.className("
        benefit-txt"));
    for (int i = 0; i < texts.size(); i++) {
        String className = texts.get(i).getAttribute("class");
        assertTrue(className.contains("benefit-txt"), "Блок #" + (i
            + 1) + " не содержит класс 'benefit-txt'");
        assertNotEquals(texts.get(i).getCssValue("display"), "none"
            , "Блок #" + (i + 1) + " скрыт");
    }
}

@Test(priority = 7)
public void testFrameExistence() {
    WebElement frame = driver.findElement(By.id("frame"));
    assertTrue(frame.isDisplayed(), "Iframe с id='frame' не найде
        н");
    String src = frame.getAttribute("src");
    assertTrue(src.contains("jdi-testing"), "src iframe должен со
        держать 'jdi-testing'");
}

@Test(priority = 8, dependsOnMethods = "testFrameExistence")
public void testFrameContent() {
    driver.switchTo().frame("frame");
    try {
        WebElement frameButton = driver.findElement(By.id("frame -
            button"));
        assertTrue(frameButton.isDisplayed(), "Кнопка в iframe не о
            тображается");
        assertEquals(frameButton.getText().trim(), "Frame Button");
    } finally {
        driver.switchTo().defaultContent();
    }
}

@Test(priority = 9)
public void testLeftMenuCount() {
    List<WebElement> items = driver.findElements(By.cssSelector("
        ul.sidebar-menu.left > li"));
    assertEquals(items.size(), 5);
}

```

```

97     }
98
99     @Test(priority = 10, dependsOnMethods = "testLeftMenuCount")
100     public void testLeftMenuTexts() {
101         List<WebElement> items = driver.findElements(By.cssSelector("
            ul.sidebar-menu.left > li"));
102         String[] expected = {"Home", "Contact form", "Service", "
            Metals & Colors", "Elements packs"};
103         for (int i = 0; i < items.size(); i++) {
104             assertEquals(items.get(i).getText().trim(), expected[i]);
105         }
106     }
107 }

```

Приложение 8. Исходный код Task2Test

```

1 package ru.truknok.browsertest;
2
3 import org.openqa.selenium.*;
4 import org.openqa.selenium.interactions.Actions;
5 import org.openqa.selenium.support.ui.ExpectedConditions;
6 import org.openqa.selenium.support.ui.WebDriverWait;
7 import org.testng.annotations.Test;
8 import org.testng.asserts.SoftAssert;
9 import java.io.*;
10 import java.time.Duration;
11 import java.util.List;
12
13 public class Task2Test extends DriverSetup {
14
15     private void saveDebugInfo(String testName, Exception e) {
16         try {
17             File src = ((TakesScreenshot) driver).getScreenshotAs(
18                 OutputType.FILE);
19             src.renameTo(new File("error_" + testName + ".png"));
20             String html = driver.getPageSource();
21             try (FileWriter writer = new FileWriter("error_" + testName
22                 + ".html")) {
23                 writer.write(html);
24             }
25         } catch (IOException ex) {
26             ex.printStackTrace();
27         }
28     }
29
30     @Test
31     public void testDifferentElementsPageFullCheck() {
32         SoftAssert softAssert = new SoftAssert();
33         String testName = "DifferentElements";
34         try {

```

```

33 WebDriverWait wait = new WebDriverWait(driver, Duration.
    ofSeconds(15));
34 Actions actions = new Actions(driver);
35
36 WebElement serviceBtn = wait.until(ExpectedConditions.
    elementToBeClickable(By.xpath("//span[text()='Service']"
    )));
37 actions.moveToElement(serviceBtn).perform();
38 serviceBtn.click();
39
40 Thread.sleep(800);
41 WebElement targetBtn = wait.until(ExpectedConditions.
    elementToBeClickable(By.xpath("//span[text()='Different
    elements']")));
42 ((JavascriptExecutor) driver).executeScript("arguments[0].
    scrollIntoView(true);", targetBtn);
43 Thread.sleep(500);
44 targetBtn.click();
45 Thread.sleep(1500);
46
47 String title = driver.getTitle();
48 softAssert.assertTrue(title.toLowerCase().contains("
    different") && title.toLowerCase().contains("element"));
49
50 List<WebElement> allButtons = driver.findElements(By.
    cssSelector("input[type='button'], button"));
51 softAssert.assertTrue(allButtons.size() >= 4);
52
53 List<WebElement> checkboxes = driver.findElements(By.
    cssSelector("input[type='checkbox']"));
54 softAssert.assertTrue(checkboxes.size() >= 4);
55
56 List<WebElement> radios = driver.findElements(By.
    cssSelector("input[type='radio']"));
57 softAssert.assertTrue(radios.size() >= 4);
58
59 List<WebElement> selects = driver.findElements(By.tagName("
    select"));
60 softAssert.assertFalse(selects.isEmpty());
61
62 WebElement dropdown = selects.get(0);
63 softAssert.assertTrue(dropdown.isDisplayed());
64
65 List<WebElement> options = dropdown.findElements(By.tagName
    ("option"));
66 softAssert.assertTrue(options.size() >= 2);
67
68 boolean hasRed = options.stream().anyMatch(o -> o.getText()
    .equalsIgnoreCase("Red"));
69 boolean hasGreen = options.stream().anyMatch(o -> o.getText
    ().equalsIgnoreCase("Green"));
70 softAssert.assertTrue(hasRed && hasGreen);

```

```
71     List<WebElement> labels = driver.findElements(By.className(  
72         "label-checkbox"));  
73     softAssert.assertTrue(labels.size() >= 2);  
74  
75     } catch (Exception e) {  
76         saveDebugInfo(testName, e);  
77         softAssert.fail("Ошибка в тесте: " + e.getMessage());  
78     }  
79     softAssert.assertAll();  
80 }  
81 }
```